Annotating single-cell data on a laptop: a 12-method benchmark and practical low-memory workflows, with actinn-jax

Ian Driver*

Abstract

Reference-based cell-type annotation is a routine, high-volume step in single-cell analysis, and most labs run it on a laptop, not a GPU cluster. We present **actinn-jax**, a from-scratch JAX reimplementation of ACTINN (a 4-layer MLP classifier) with a sparse-aware pipeline and a train-once/map-many cached reference model, and use it as the occasion for a neutral benchmark of **twelve** methods spanning every major class — classical (SVM, kNN), regularized-linear (CellTypist), a carefully-tuned linear pipeline, parameter-free projection (scTOP), marker/correlation (SingleR, scmap), deep probabilistic (scANVI, scArches), an interpretable prototype VAE (ProtoCloud), and a foundation model (scPRINT) — across **six** datasets (within-dataset, cross-dataset and cross-study; 8 to 86 cell types) plus a five-point **scaling sweep to 49k reference cells**, measuring accuracy, macro-F1, ontology-aware concordance, training and inference time, and peak memory on commodity Apple-Silicon hardware. **Accuracy differences among the leading methods are small; their cost differences are not.** The top of the accuracy table is a five-way cluster spanning 0.008 (0.831–0.839), led by a carefully preprocessed **linear pipeline** (normalize → ANOVA → PCA → logistic regression, 0.839) rather than by a deep model, with scANVI, scArches, actinn-jax (0.831) and CellTypist alongside it. **Those same five methods differ by ~200× in inference time** (0.35 s to 73 s) **and 2.7× in peak memory** (1.6 to 4.3 GB). The linear pipeline is the most accurate-per-second option on laptop-sized data, fitting 4× faster than actinn-jax (4.0 vs. 16.0 s). At atlas scale the ordering changes: **ProtoCloud** rises from worst at 3k reference cells (0.722) to best at 49k (**0.976**, vs. 0.936 for actinn-jax and 0.939 for the linear pipeline). No method leads everywhere, and methods separated by less than a percentage point of accuracy are separated by two orders of magnitude in cost. Because several methods now annotate quickly, accurately, and with a usable handle on unknown cells, the useful contribution is not another ranking but an account of **which tool fits which job, and what the surrounding workflow looks like**. We report the benchmark as a decision aid (accuracy-per-second, accuracy-per-byte, and behaviour from 3k to 49k reference cells), then use actinn-jax to demonstrate **end-to-end workflows that a low, flat cost profile makes practical**: annotating an unknown human dataset from a **shipped census-scale reference** (~800 cell types, calibrated abstain) with no training; **tissue-aware refinement** that halves spurious cross-tissue labels on a liver query; a **broad→refined hand-off** to a small focused reference (held-out cross-study liver 0.23/0.58 → 0.72/0.86, exact-CL/ontology); within-cell-type resolution (hepatocyte zonation); and a **cluster-level novel-cell-type screen** that recovers a withheld pulmonary ionocyte population and its marker ASCL3. What enables this is inference that is **sub-second and flat** — independent of reference size and cardinality — over a cached reference that is trained once and reused, at the lowest peak memory of the methods we carried to atlas scale (6.1 vs. 13.2 GB for the linear pipeline at 49k cells). The components are not unique: ProtoCloud provides uncertainty, gene attribution and a retraining-based refinement path, and **Pan-human Azimuth** ships a pretrained, calibrated, hierarchical pan-human annotator that runs on a laptop — a better-resourced broad tier than ours, and one we can **distil into a tier-1 model of comparable accuracy at ~9× its throughput, built with no GPU and no labeled data**. What the workflow adds is the

hand-off into a label set the broad model was never trained on: refinement against a user's own focused reference, and resolution below any fixed typology's leaves.

Independent Researcher, Detroit, MI, USA
* Correspondence: driver.ian@gmail.com

1 Introduction

Annotating cell types by mapping to a labeled reference is one of the most frequently run operations in single-cell RNA-seq. The method landscape is large (survey), but published comparisons ([Abdelaal 2019], [Huang 2024]) emphasize accuracy and rarely report the axis that decides what a working scientist actually runs on their own machine: **wall-clock time and memory on commodity hardware**, without a GPU. Foundation models (scGPT, Geneformer, scPRINT) push accuracy in some settings but need GPUs and minutes-to- hours, and — as we and others find — their *zero-shot* label predictions underperform a small model trained on curated reference labels.

We ask a practical question: **for a given annotation job on commodity hardware, which method should you actually run, and what does the surrounding workflow look like?** Several methods now annotate quickly and accurately and give a usable signal on unknown cells, so the shortage is not another leaderboard but guidance on fit-for-purpose — accuracy per second, accuracy per byte, and how each behaves as the reference grows. We answer with a neutral 12-method benchmark plus a set of demonstrated workflows, held to a single standard: every method runs in its own environment through the same harness, on the same splits, scored by the same metrics, and reported as measured — including where actinn-jax is not the leader.

Contributions. 1. A modern, dependency-light (no TensorFlow) JAX reimplementation of ACTINN with sparse preprocessing, chunked atlas-scale prediction, and a cached reference model. 2. A neutral **12-method** × **6-dataset** benchmark of accuracy, speed, and memory on Apple Silicon, with a rejection/abstain analysis and a **five-point scaling sweep to 49k reference cells**. The panel deliberately includes the baselines most likely to beat a small MLP — a tuned linear pipeline, scTOP, and ProtoCloud — and the results are reported as measured (§3.1, §5). The sweep also surfaced the slowest fit in the study as ours (359 s on a 47k-cell, 32k-gene liver reference), which profiling traced to an unconditional per-batch host sync and ACTINN's inherited fixed batch of 128 (~24k tiny update steps at that size). Both are fixed upstream — the batch now scales with the reference — giving **1.8–2.1× faster fits at 20k–49k cells with accuracy unchanged**, and leaving references below ~12.8k cells bit-identical. 3. **Demonstrated end-to-end workflows** for jobs where cost is the binding constraint, all CPU: annotate an unknown human dataset from a **shipped census-wide ~800-type reference** with a calibrated abstain and no training; **tissue-aware refinement** (halves spurious cross-tissue labels on a liver query); a **broad→refined hand-off** to a small focused reference (cross-study liver 0.23/0.58 → 0.72/0.86); within-cell-type resolution (hepatocyte zonation); and a **one-call novel-cell-type screen** (recovers a withheld pulmonary ionocyte population and its marker ASCL3). We are precise about what is not ours: ProtoCloud offers uncertainty, attribution and a retraining-based refine, and Pan-human Azimuth ships a pretrained hierarchical pan-human annotator with trained abstention, so the broad tier itself is not a distinguishing feature — we distil that model into a tier-1 reference rather than compete with it. What is distinct is **refinement into a label set no pretrained model carries** — the user's own focused reference, and states below a fixed typology's leaves. 4. An **independent external validation** on Open Problems `label_projection`, with a controlled same-hardware speed/memory comparison, and a set of cheap ablations that characterize *when* the

gene-space model gains or loses to heavier methods (input standardization; a reference-guided gene budget; a foundation-model negative control).

2 Methods

2.1 actinn-jax

actinn-jax reimplements ACTINN ([Ma & Pellegrini 2020]) — a 4-layer fully-connected network (100/50/25 hidden units, ReLU, softmax) trained with Adam — in JAX/optax, replacing the original's TensorFlow-1.x graph/session code. Key engineering: - **Sparse-aware preprocessing**: no `adata.to_df` densification; CP10k+log2 normalization and the expr/CV gene filter run on sparse matrices; only the selected-gene columns of each minibatch are densified. - **Cached reference model** (`ReferenceModel`): train once, `save`/`load`, map many queries — the amortized cost of repeated annotation against a fixed reference drops to the inference cost alone. - **Chunked prediction** for atlas-scale queries; `use_raw='auto'` to pick raw counts from `.raw` (CELLxGENE convention). - **Optional reference-fit standardization** (`standardize=True`): z-score each selected gene by the reference's frozen mean/std and apply it to the query — a cheap domain alignment (§3.8). Opt-in, with a saved scaler that projects with the model; μ/σ persist in the `.npz` (old models load standardization-off, backward-compatible). The reimplementation reproduces the original's accuracy to within repeat noise while being **substantially faster and lighter and, unlike the TensorFlow-1.x original, installable on current Python/ML environments** — the original's `tf.compat.v1` graph/session code no longer runs on a modern stack without pinning a years-old TensorFlow. Because the two are the same model at equal accuracy, we do not carry the original as a separate benchmark row; the engineering win is the point, not an accuracy comparison.

2.2 Benchmarked methods

method	tier	engine	rejection	env
actinn-jax	classical	JAX MLP	abstain (min_prob)	core
SVM	classical	linear SVM (SGD)	—	core
kNN	classical	k-nearest neighbors	—	core
CellTypist	linear	L2 logistic regression	prob threshold	core
linear-anova-pca	linear	normalize→ANOVA→PCA(220)→logreg	prob	core
scTOP	parameter-free	rank z-score class-average projection	—	core (`sctop`)
SingleR	correlation	Spearman + fine-tuning	—	R/.Rlib
scmap-cluster	correlation	centroid cosine	yes (unassigned)	R/.Rlib

method	tier	engine	rejection	env
scANVI	deep	scVI semi-supervised VAE	prob	.venv-scvi (MPS)
scArches	deep	scANVI reference surgery	prob	.venv-scvi (MPS)
ProtoCloud	deep	prototype VAE + LRP attribution	ambiguity flag	.venv-protocloud
scPRINT	foundation	pretrained transformer, zero-shot	—	.venv-scprint (MPS)

Every method above except scPRINT runs the full accuracy/speed/memory matrix of §3.1–§3.2 on all six datasets, so no method is compared on only a subset. Two are deliberately outside that matrix, each for a stated reason: - **scPRINT** is a zero-shot foundation model with a *fixed* Cell-Ontology label vocabulary; it cannot emit labels outside that set and skips symbol-keyed datasets, so it is not a drop-in reference classifier. We report it separately as the "just use a foundation model" reference point (§3.7, and externally in §3.8). - **scPred** is excluded outright: unmaintained; it requires harmony's removed `HarmonyMatrix` API and older harmony versions do not compile on the current toolchain.

2.3 Datasets

dataset	split	tissue	#types	genes	notes
lung_intra	within-dataset	lung (Krasnow)	46	Ensembl	300 cells/type ref
lung_cross	cross-dataset	lung (HCLA→Krasnow)	46	Ensembl	different lab/protocol
liver_intra	within-dataset	liver (HLiCA)	36	Ensembl	150 cells/type
liver_cross	cross-**study**	liver (HLiCA)	34	Ensembl	train 6 studies → test withheld study
blood_gut_intra	within-dataset	blood+gut (Sanger)	86	Ensembl	high cardinality; no CL ids
pbmc	within-dataset	PBMC (pbmc3k)	8	symbols	small-n; scPRINT skips (symbols)

The set spans the three generalization regimes (within-dataset, across datasets, across studies), an order of magnitude in cell-type count (8→86), and both gene-ID conventions.

2.4 Metrics & protocol

Per (dataset, method, repeat): **accuracy**, **macro-F1** (unweighted over classes, so rare types count), **ontology-aware concordance** (a call is correct if it is the same node, an ancestor, or a descendant of the truth in the Cell Ontology — credits right-lineage-wrong-granularity calls; only where both ref and query carry CL ids), **fit time**, **predict time**, **peak RSS**. Each method runs in its own venv via subprocess isolation (`benchmark/runner.py`); the driver (`benchmark/driver.py`) builds identical ref/query splits once per dataset and scores every method against them. **repeats = 3** (scPRINT: 1, near-deterministic and embed-dominated).

Hardware: Apple Silicon (M-series), CPU for the classical/linear/correlation tiers and actinn-jax; Apple MPS for the deep and foundation tiers. No discrete GPU, no cloud — the regime the benchmark is about. Math-library threads are left uncapped (wall-clock as a practitioner sees it); an optional thread cap is available for stricter reproducibility.

3 Results

3.1 Accuracy across datasets

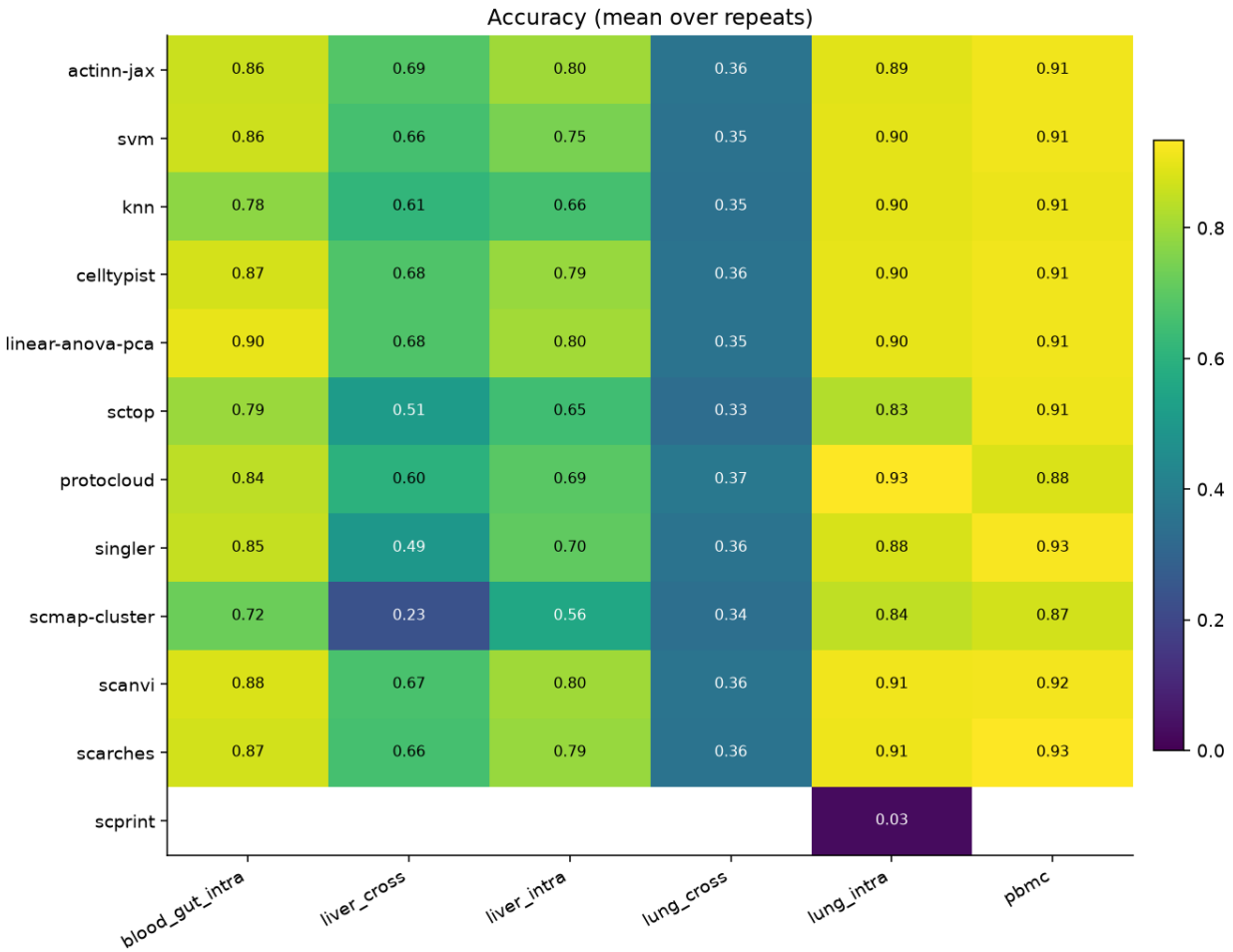

Figure 1: Accuracy by method (rows) and dataset (columns), in-house panel.

Mean accuracy across the **five shared-vocabulary datasets** (lung_cross excluded — its exact accuracy is a vocabulary artifact, see the † note below; means over all six are ~0.08 lower for every method with the identical ranking):

method	mean acc (5 datasets)	mean macro-F1 (6)	mean ontology (6)
linear-anova-pca	**0.839**	0.699	0.808
scANVI	0.835	**0.701**	**0.812**
scArches	0.833	0.700	0.809
actinn-jax	0.831	0.683	0.811
CellTypist	0.831	0.697	0.807
SVM	0.816	0.688	0.797
ProtoCloud	0.790	0.649	0.778
SingleR	0.770	0.652	0.750
kNN	0.770	0.624	0.769
scTOP	0.739	0.619	0.703
scmap-cluster	0.646	0.550	0.771

The top of the table is a **five-way tie within 0.008** — the tuned **linear pipeline (0.839)**, scANVI (0.835), scArches (0.833), actinn-jax (0.831) and CellTypist (0.831) — led by the linear pipeline rather than by a deep model. actinn-jax is essentially tied for the best ontology-aware concordance (0.811 vs scANVI's 0.812). ProtoCloud (0.790) and scTOP (0.739) sit below that cluster on these subsampled references: both need conditions these splits do not provide — ProtoCloud is data-hungry and becomes the strongest method once given a real atlas (§5, `SCALING_MEMORY.md`), while scTOP is built for small, low-cardinality problems. Per dataset (accuracy):

dataset	actinn-jax	best method (acc)
lung_intra (46 types)	0.894	ProtoCloud 0.932
lung_cross (cross-dataset)†	0.358 exact / 0.749 ontology	ProtoCloud 0.374 / **0.791 ontology**
liver_intra (36 types)	0.802	linear-anova-pca 0.804 (actinn-jax #2)
liver_cross (cross-study)	**0.686 exact / 0.731 ontology — #1 on both**	actinn-jax
blood_gut (86 types)	0.860	linear-anova-pca 0.902
pbmc (8 types)	0.913	scArches 0.934

No single method leads everywhere: the linear pipeline and ProtoCloud each take two datasets, actinn-jax and scArches one apiece. actinn-jax leads the cross-study liver split — the hardest generalization regime here, and the one closest to real reference mapping — and is second on within-dataset liver, while trailing on lung, on the 86-type blood+gut set (−4.2 pt to the linear pipeline), and on small-n pbmc. The spread across the leading methods on any one dataset is 1–4 points. scmap-cluster is consistently the weakest (0.24 macro-F1 on liver_cross).

† lung_cross exact accuracy (~0.35 for every method) is a label-vocabulary artifact, not a transfer failure. HCLA (reference) and Krasnow (query) were annotated independently

and share only **20 of 46 cell-type names**; 26 of Krasnow's types are absent from HCLA's vocabulary, so no classifier can emit the exact string. The mismatch is granularity: HCLA's finer taxonomy has `alveolar / elicited / lung macrophage` but **no generic `macrophage`** (CL:0000235), while Krasnow labels many cells generic `macrophage` — so an HCLA-trained model predicts `lung macrophage` (CL:1001603) for them, exact-wrong but ontology-correct (*lung macrophage is-a macrophage*); likewise Krasnow's generic `endothelial cell` (CL:0000115) has no HCLA counterpart. **Ontology-aware concordance — which credits a same-lineage call — is ~0.75 for the trained methods (0.67–0.79 across the full panel, highest for ProtoCloud)**, so cross-dataset transfer actually works; exact-match just conflates classifier error with vocabulary/granularity mismatch. This is precisely why we report ontology concordance, and it is a general benchmarking pitfall: **cross-dataset exact accuracy between independently-annotated atlases is not a meaningful accuracy signal.** The other five datasets use a single shared label vocabulary for reference and query (a split of one atlas, or HLiCA's harmonized annotations), so their exact scores are unaffected.

3.2 Speed and memory

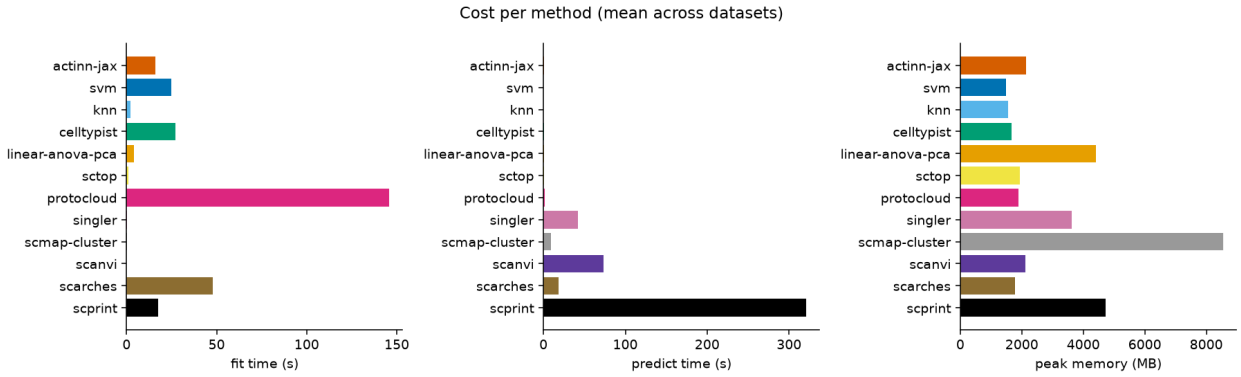

Figure 2: Per-query inference time and peak memory by method (in-house panel).

The accuracy tie hides a two-order-of-magnitude spread in cost. Mean per query across datasets:

method	fit (s)	**predict (s)**	peak mem (MB)
SVM	24.9	**0.10**	1480
kNN	2.3	**0.24**	1546
linear-anova-pca	**4.0**	**0.35**	4403
actinn-jax	16.0	**0.37**	2145
CellTypist	27.0	**0.63**	1661
scTOP	1.0	1.08	1928
ProtoCloud	145.9	1.42	1895
scmap-cluster	0.2	9.0	8550
scArches	47.7	18.7	1783
SingleR	0.2	42.4	3621
scANVI	0.0*	73.1	2118

(*scANVI does most of its work in a single train+predict pass, attributed to predict.) Five methods — SVM, kNN, the linear pipeline, actinn-jax, CellTypist — predict in well under a second; scTOP

and ProtoCloud are ~1–1.4 s; the deep/correlation tiers are 25–200× slower. actinn-jax predicts in **0.37 s vs. scANVI's 73 s at statistically tied accuracy** (~195×), and is far lighter than the memory-heavy tiers (scmap 8.5 GB, SingleR 3.6 GB).

3.3 The accuracy × speed frontier

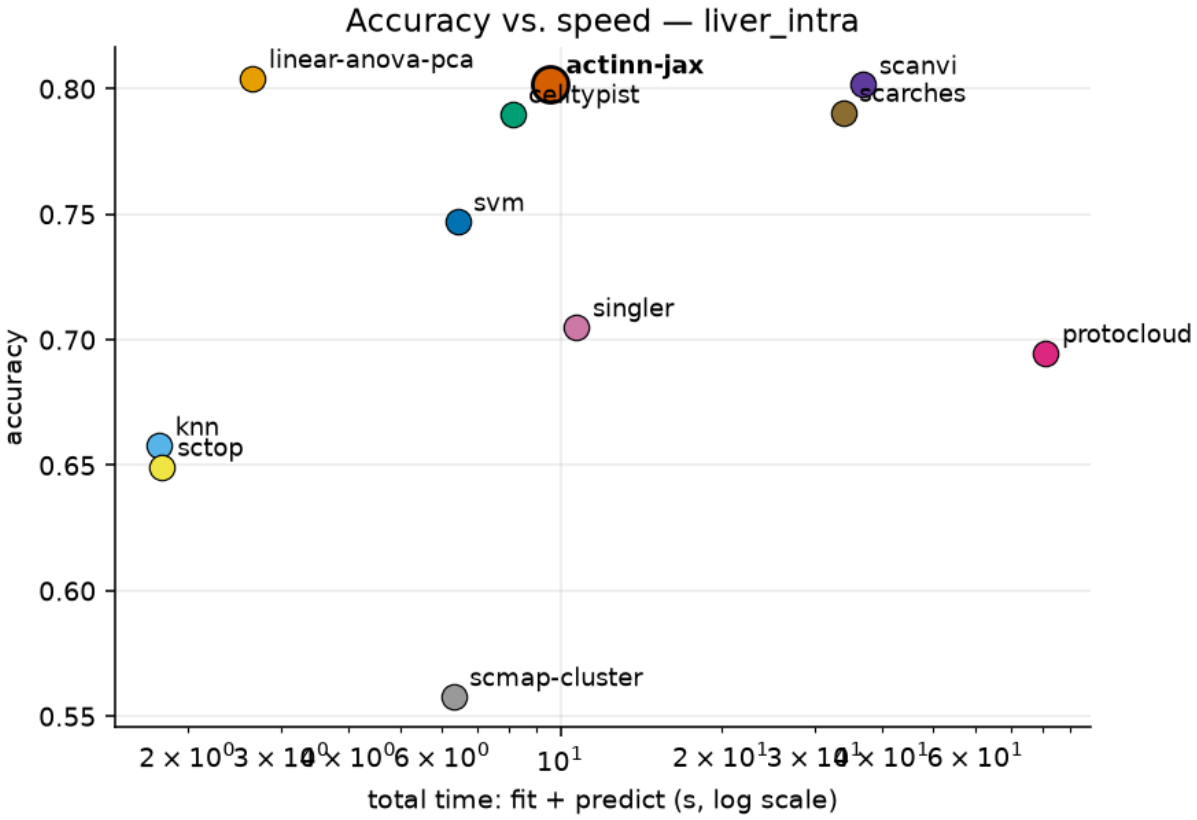

Figure 3: Accuracy versus total wall time (liver_intra); actinn-jax sits on the fast frontier.

Plotting accuracy against total wall time makes the frontier explicit. The **tuned linear pipeline** holds the fast-frontier point: highest matrix accuracy (0.839) at the lowest fit time of any accurate method (4.0 s), placing it above and to the left of actinn-jax, SVM, kNN and CellTypist. The deep methods buy little or nothing on accuracy here — at or below the linear pipeline on every dataset except lung — for 50–200× the inference cost. actinn-jax sits just inside the frontier at this reference size; what separates it is not visible on a fixed-size plot but on the scaling axes of §3.4 and §5: predict time that stays flat as the reference grows, and ~2× lower peak memory that holds to atlas scale. The figure is drawn from the liver_intra panel and depicts the ranking in the tables above.

3.4 Scaling

Training time grows with reference size and with #cell types for all trained methods — actinn-jax's fit goes 13 s → 141 s → 254 s as the reference grows 965 → 14.8k → 24k cells, comparable to CellTypist (4 s → 75 s → 246 s) and somewhat above SVM (9 s → 55 s → 80 s); kNN is lazy (fit is trivial but it pays at predict and is the least accurate).

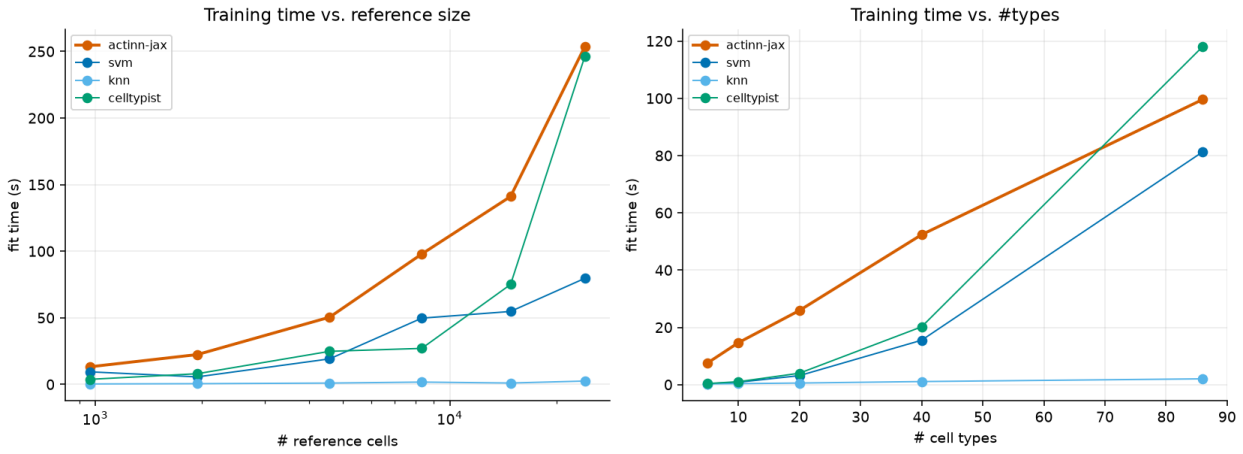

Figure 4: Fit and predict time versus reference size and cardinality; predict time stays flat and sub-second.

The other axis is the one that matters for the workflows: **predict time stays flat and sub-second across the entire range — ~0.3–1.4 s whether the reference has 1k or 24k cells, or 5 or 86 types.** Inference cost does not grow with reference size or cardinality, and with the train-once/map-many cache the fit cost is paid once and the flat predict cost is all that recurs — the regime that matters when a reference is reused across many queries. (The scaling script's memory column is process-cumulative and not a clean per-size measurement; per-method peak memory is reported from the isolated-subprocess matrix in §3.2.)

3.5 The large→refined annotation workflow

What actinn-jax is built around is not a single classifier but a **two-tier workflow** that matches how annotation is actually done: get a broad call fast, then sharpen it where it matters. Because every stage is a cached `ReferenceModel` with sub-second, memory-bounded inference (§3.2, §3.4), the whole workflow runs on a laptop.

Tier 1 — broad, census-scale. A shipped ~800-type human reference built from the CELLxGENE census gives any query a first-pass annotation across the whole body, with a **calibrated abstain threshold** so out-of-reference cells are flagged rather than force-labeled (see the smooth abstain curve of §3.6). This is the "what am I looking at" pass — broad coverage, deliberately not fine-grained.

Tier 1 can be built from a model instead of from labels. The census route above needs a foundation model on a GPU to discover its hierarchy. A pretrained pan-human annotator already has one — Pan-human Azimuth publishes an 8-level typology with every node mapped to a Cell Ontology term — so labelling a corpus with it and training actinn-jax on those labels transfers both the vocabulary and the structure. That build needs **no accelerator and no labeled input**, only raw human counts: under ten minutes of CPU on 85k cells drawn from three atlases plus a census-wide sample. On 3,396 withheld cross-study liver cells:

tier-1 model	classes	ontology	cells/s
census-built (`broad_human_v1`)	798	0.338	2,962
Pan-human Azimuth	382	0.380	1,076–1,563
distilled (`panhuman_distill_v1`)	324	**0.406**	**8,937–10,021**

This makes the entry point both better and ~3× faster than building it from the census directly, and it answers in a harmonized, ontology-mapped vocabulary rather than raw census strings. Withholding a whole atlas from the corpus, the distilled model tracks the one it was distilled from to within **1.5 points** on lung (0.695 vs 0.710) and **3.0** on liver (0.481 vs 0.511) — the corpus, not the recipe, is what bounds it. Two caveats keep this modest. The 0.406/0.380 ordering is one query, and both actinn-jax models draw on a census sample that may include these studies while Pan-human Azimuth does not, so read the distilled model as *comparable to* the one it distils rather than better. And it does **not** inherit that model's trained abstention: its confidence separates right from wrong poorly (90.5% coverage at `p ≥ 0.5` moves concordance only 0.406 → 0.427), so the calibrated tier-1 abstain of §3.6 belongs to the census-built reference until this one is recalibrated.

Tier 2 — refined, tissue-focused. For the tissue the broad pass identifies, a small focused reference re-annotates at full resolution. The gain is large and it is the crux of the workflow: on held-out **cross-study** liver cells, the broad census model scores exact-CL **0.23** / ontology **0.58**, while a focused **48-type HLiCA liver** reference on the *same cells* reaches **0.72 / 0.86**. Refinement is where fine-grained accuracy comes from; the broad model's job is to route to it, not to be right about subtypes itself.

The two tiers switch; they do not stack — we tested this and it is worth stating plainly, because the opposite is the natural assumption. On the leakage-free cross-study liver split, substituting the stronger **Pan-human Azimuth** as tier 1 lifts the broad pass (ontology 0.380 vs 0.338 for our own broad model) but changes nothing downstream. Using that tier-1 call to *narrow* tier 2's classes — the zero-retrain masking actinn-jax ships — makes the result **worse**, 0.731 → 0.708: tier 1's coarse call matches the true lineage on 85.8% of cells, and the 14% it misses cost more than the 86% it gets right can gain, since a wrong mask discards the correct class outright. A **perfect** coarse call would add only **+2.8 points** (0.759). Once tier 2 covers the tissue, there is almost no accuracy left for a broad model to contribute; its value is choosing which focused reference to load and covering cells none of them claims (`PAN_HUMAN_AZIMUTH.md`).

Supporting mechanisms. (i) A **foundation-model-guided coarse→fine hierarchy**: a foundation model discovers a coarse→fine structure *offline*, the fast CPU model uses it at inference — beats a flat classifier on 3/4 datasets, matches an expert hierarchy, beats a random-grouping control, and never depends on the foundation model at prediction time. (ii) **Within-cell-type resolution**: the same machinery resolves hepatocyte zonation (portal→central) at ~0.99 within-one-zone, generalizing across donors and datasets — a third tier below the cell-type label. Together these make actinn-jax a *pipeline* (broad → tissue → subtype/state), not just a classifier, at classical-method speed throughout.

3.6 Rejection / abstain

Holding out 9 of 36 cell types entirely from the HLiCA liver reference (so 1,350 query cells are genuinely out-of-distribution), and sweeping a confidence threshold:

threshold	actinn-jax: acc(kept) / coverage / OOD-flagged	CellTypist: acc(kept) / coverage / OOD-flagged
0.0	0.885 / 1.00 / 0.00	0.873 / 1.00 / 0.00
0.5	0.919 / 0.93 / 0.30	0.936 / 0.68 / 0.68
0.7	0.946 / 0.84 / 0.52	0.936 / 0.68 / 0.68
0.9	0.969 / 0.66 / 0.73	0.937 / 0.68 / 0.68

actinn-jax's probability gives a **smooth, tunable** abstain curve — accuracy on kept cells rises 0.885→0.969 as coverage falls 1.00→0.66 and OOD-flagging climbs 0.00→0.73 — so a user can dial the precision/coverage/novel-cell-detection trade-off. CellTypist's probabilities are **saturated** (near 0 or 1): the threshold jumps to 68% OOD-flagged at 0.5 and then does not move across 0.5–0.9, giving essentially one operating point rather than a tunable curve. scmap-cluster offers a single native "unassigned" decision (no sweep). A tunable threshold is what makes the abstain step usable as a routing decision in the workflow of §3.5.

3.7 Foundation-model zero-shot (scPRINT)

scPRINT is run separately as a zero-shot predictor (no training on the reference), as a reference point for the "just use a foundation model" alternative. On the lung datasets it scored **exact accuracy 0.027 / ontology concordance 0.206 in 321 s per query** — vs. actinn-jax's 0.89 in 0.2 s — so as a *label* predictor it is both slow (~1000× actinn-jax) and weak. Its pretrained classifier head also carries a **fixed label vocabulary** and requires CL ids, so it cannot emit labels outside that set; this is an inherent property of a zero-shot label head, not a defect, but it means the foundation model's raw predictions are not a drop-in annotator. This is exactly why our two-stage workflow uses scPRINT's **embeddings** — its learned structure — to shape a small trained model, never its zero-shot labels: the foundation model is valuable, its raw label predictions are not.

3.8 External validation: Open Problems `label_projection`

Our in-house benchmark is neutral but self-run. For an **independent** check, we ran actinn-jax through the community-standard Open Problems label-projection task — their 6 datasets, splits, and metrics — and slotted it into their published v2.0.0 leaderboard.

Placement. On a benchmark we did not design and cannot tune, **actinn-jax places 3rd of 17 on mean accuracy (0.837), 1st among all methods that complete every dataset**, beats its PCA-space `mlp` sibling, and posts the best accuracy of any completing method on the hardest dataset (tabula_sapiens, 160 types: 0.394 vs mlp 0.342). It is upper-mid on macro-F1 and **not the top method overall** — scANVI+scArches surgery and xgboost score higher on the datasets they finish, though both **fail to complete tabula_sapiens**, which is why they rank above actinn-jax only on a mean over the 5 easier datasets. The foundation models again land at the bottom (scgpt_zeroshot 0.639, uce 0.131 ≈ the random-labels control), reproducing §3.7 externally.

Controlled same-hardware speed & memory. Leaderboard runtimes mix hardware, so we re-ran actinn-jax and the CPU tier through OP's *own* Nextflow pipeline on a **single AWS**

`r7i.8xlarge` — identical hardware, harness, storage, and trace instrumentation for every method. This is the paper's cleanest cross-method cost comparison:

method (same box)	mean acc	macro-F1	runtime/dataset	peak RSS
actinn-jax	0.837	0.655	**165 s**	21.0 GB
mlp	0.838	0.664	327 s	19.9 GB
logistic_regression	0.813	0.689	29 s	20.0 GB
knn	0.793	0.648	11 s	19.5 GB
xgboost	0.795	0.613	905 s	80.7 GB
cellmapper_linear	0.776	0.553	58 s	31.5 GB
naive_bayes	0.738	0.613	31 s	19.5 GB

actinn-jax **ties its `mlp` sibling on accuracy at ~2× the speed** (165 vs 327 s) and **beats xgboost's accuracy at ~5.5× less runtime and ~4× less memory** (165 s / 21 GB vs 905 s / 81 GB). The faster methods (knn, logistic) are less accurate; the more accurate heavyweight (xgboost) is far slower and heavier — Pareto-non-domination **within Open Problems' method set**, on controlled hardware. That set contains no carefully-tuned linear pipeline, and where we add one ourselves it leads on accuracy and fit time (§3.1, §3.2), so this is a statement about the OP panel rather than a general one.

Extension: our four added methods on the same harness. We later ported the tuned linear pipeline, scTOP, the SGD-SVM and CellTypist into Open Problems components and ran them through the same pipeline on a fresh `r7i.8xlarge` (`results_openproblems_samehw_v2.csv`, raw trace archived alongside). All four run correctly on a harness we did not write, which is the main thing this establishes; peak memory sits in the same 13–20 GB band as the incumbent methods (xgboost is the outlier at 55 GB). **We do not report cross-method runtimes from this run, and the reader should not infer them from the CSV.** Two confounds make the wall-clock numbers non-comparable: the run used a higher task concurrency (6 vs 2), and the methods differ enormously in how many cores they take — measured `%cpu` ranges from **49% for CellTypist (half a core) and 117% for `mlp` (effectively single-threaded) to 2338% for the linear pipeline (23 cores)**. Wall-clock under those conditions measures threading and contention as much as algorithmic cost. It also disagrees with the controlled run above for two methods (`mlp` 327 s → 769 s, `cellmapper_linear` 58 s → 717 s), which we flag rather than reconcile: the table above, run at low concurrency, remains the cost comparison we stand behind. Coverage was partial when a wall-clock budget stopped the run — six datasets for the SVM and CellTypist, four for scTOP, three for the linear pipeline — so per-method means over different dataset subsets would not be comparable either.

Three cheap ablations, and their limits. (i) *Input standardization* — z-scoring genes to the reference's frozen mean/std, a CPU-only domain-alignment inspired by scArches reference surgery — lifts mean accuracy +0.2 and macro-F1 +1.2 pt (largest on batch-shifted datasets: gtex macro-F1 +7.9). It is shipped **opt-in** (`standardize=True`) because it shifts the softmax calibration that the two-stage abstain thresholds are tuned against. (ii) *Gene budget* — OP feeds every method 1000 HVGs, which most starves a gene-space MLP. Widening to ~5000 HVGs lifts accuracy on 4 of 6 datasets (immune/gtex +3 pt) and there **matches scANVI+scArches's full-config leaderboard accuracy** (immune 0.891 ≈ 0.892) on CPU in minutes — the gap to the top method was largely a gene-budget artifact, not the VAE. But more genes is **not** a universal win: it *regresses* tabula_sapiens by ~10 pt (its 284-cell test batch across 160 fine types

overfits reference-specific genes) and saturates hypomap. Crucially, this is **selectable without test labels**: held-out *reference* cross-validation rises for the datasets that benefit and is the one signal that *drops* for tabula_sapiens, and a trivial query-cells-per-class check independently flags it — so the budget can be set deterministically per dataset. (iii) *Negative control* — a CPU-only, UCE-style protein-embedding featurization (expression-weighted mean of ESM2 gene embeddings) does **not** help and hurts the hardest case: the pooling discards the per-gene detail that separates fine types, so the value of a foundation model like UCE stays locked in its GPU transformer, not a portable averaging trick.

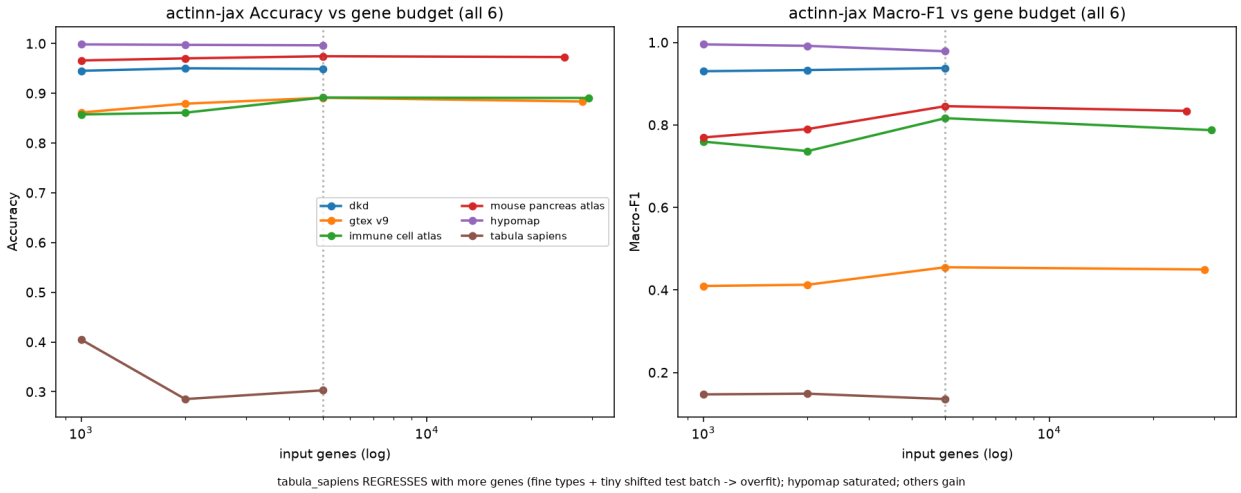

Figure 5: actinn-jax accuracy and macro-F1 versus input gene budget across all six Open Problems datasets. More genes help most datasets but regress the fine-grained, domain-shifted tabula_sapiens.

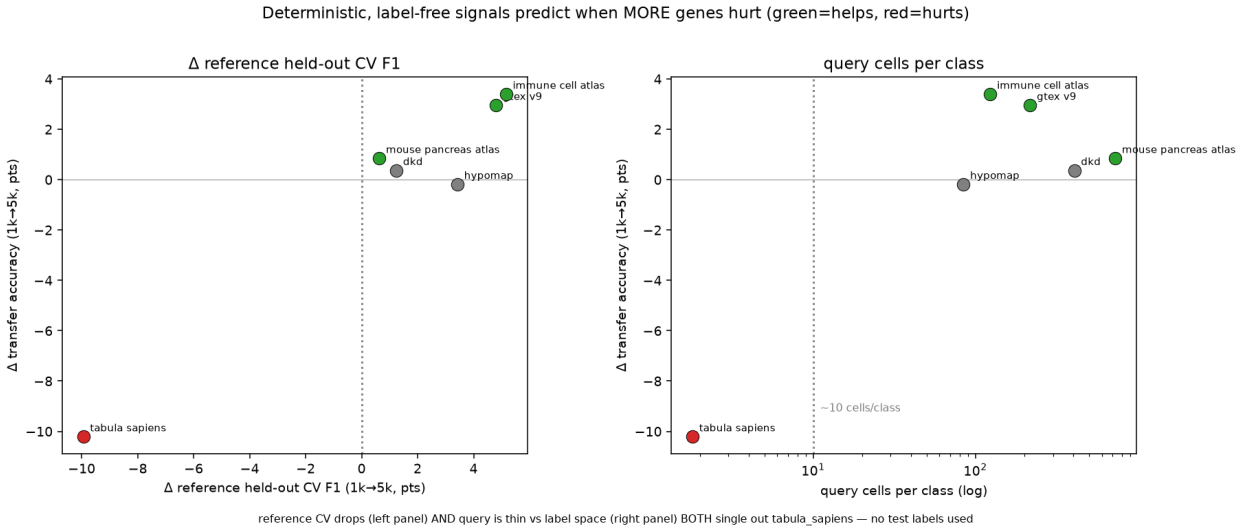

Figure 6: Label-free signals for setting the gene budget without test labels. Held-out reference cross-validation and query-cells-per-class both single out tabula_sapiens (where more genes hurt).

4 Discussion

Method choice is now dominated by cost, not accuracy. The five leading methods are separated by 0.008 in mean accuracy and by ~200× in inference time (§3.1, §3.2), so for most annotation jobs the decision is made on the cost axes. On laptop-sized references the tuned linear pipeline is the best of them on accuracy-per-second (0.839, fitting in 4.0 s); at atlas scale ProtoCloud is the most accurate by a clear margin (0.976 vs 0.936 at 49k lung cells; 0.905 vs 0.824 at 47k liver cells), having been below the cluster on the subsampled matrix. actinn-jax's place in that picture rests on a cost profile with two properties that matter for repeated use rather than for a single labelling run.

Footprint. Inference is **sub-second and flat** — independent of reference size and cardinality (§3.4) — so a query costs the same against a 1k-cell reference and a 49k-cell one. Peak memory is unremarkable on small references (2.1 GB mean — seven of the ten other methods are lighter), but it is the lowest of those we carried to **atlas scale**: ~2× below the linear pipeline (6.1 vs 13.2 GB at 49k lung cells; 6.5 vs 12.6 GB at 47k liver cells), and below scTOP above ~25k cells, where scTOP's rank processing densifies and crosses over (9.3 vs 6.5 GB at 47k). The advantage is bounded at ~2–3× rather than widening. Combined with the train-once/map-many cache, a reused reference pays fit once and then only the flat predict cost, where the linear pipeline refits scaler/PCA/classifier for every query.

Those two properties are what the workflows are built on. The practical gains come not from the flat classifier but from the **large→refined** pipeline (§3.5): a census-scale broad model with a calibrated abstain routes a query to a small focused reference that re-annotates at full resolution (cross-study liver 0.23/0.58 → 0.72/0.86), with zonation as a further tier. The pipeline is only usable because each hop is cheap — a flat sub-second call against a cached model, at bounded memory — so several models can be chained, and a large reference kept resident, on a laptop without a GPU. A method that is a few points more accurate but pays a full fit per query, or a multi-gigabyte dense expansion per stage, does not compose the same way.

Where it loses, and why. actinn-jax trails on small references and very-few-cells-per-type sets (§3.1, §5), and — a finding from the OP analysis — it is sensitive to input budget under domain shift: more genes help most datasets but overfit a tiny, fine-grained, shifted query (tabula_sapiens). The two cheapest levers are principled and label-free: **standardization** (a scArches-style domain alignment) and a **reference-CV-guided gene budget** both improve the model without peeking at test labels, and at a fair gene budget the gene-space MLP *matches the leaderboard's top method* on the datasets both complete — i.e. much of the apparent gap to heavier methods is representation budget, not model class.

Foundation models. Their **zero-shot labels** are the weakest option in both benchmarks (scPRINT, scGPT, UCE ≈ random); their **embeddings/structure** are where the value lies, and our two-stage hierarchy uses exactly that. A cheap CPU shortcut to a foundation-model representation (protein-embedding pooling) did *not* transfer (§3.8), underscoring that the value is in the trained transformer, not a portable trick. The concurrent **Pan-human Azimuth** effort reaches the same conclusion from a far larger curated corpus: a 7M-parameter supervised hierarchical MLP over a 5,055-gene panel yields cleaner annotations than scGPT and SCimilarity, whose labels they find fragmented and mis-transferred. Their stated theme — that training-data quality and organization matter as much as architecture or scale — and their finding that accuracy **saturates past ~5M training cells** (`PAN_HUMAN_AZIMUTH.md`) are independent support, from a group with no stake in our conclusion, for both halves of our reading.

When extra expressivity pays. Two lines of evidence bear on this. First, **ProtoCloud** — a prototype-based, self-explaining VAE with built-in uncertainty and gene-level attribution, a strictly richer model than ours — splits by regime rather than by a uniform margin: below the top cluster on the subsampled matrix, ahead on lung, and the strongest method at atlas scale (§3.1, §5), at ~9× actinn-jax's CPU fit time (146 vs 16 s mean) (`PROTOCLOUD.md`). The richer model pays off where it has the data to support it, and not before — which is an argument for matching model capacity to reference size, not for preferring small models everywhere. Second, and more broadly, Souza & Mehta report that **parameter-free linear representations** match or exceed single-cell foundation models across cross-species transfer, human cell-type classification and disease-state prediction — on Tabula Sapiens 2.0 their normalize→ANOVA→PCA→logistic-regression pipeline reaches mean macro-F1 0.899 against 0.907–0.910 for TranscriptFormer variants — and they offer a geometric explanation: the biologically realized cell manifold is well approximated by a **linear subspace**, so once noise is suppressed performance *saturates* and extra expressivity buys little. That is a principled account of why a four-layer MLP over normalized gene space stays competitive, and it matches our own finding that the residual gap to heavier methods was mostly **representation budget, not model class** (§3.8). Their further result that simple methods do *best* out-of-distribution — novel cell types and unseen organisms — also rhymes with the rejection behavior we depend on (§3.6).

Practical guidance. For a **one-off annotation** on laptop-sized data, use the **tuned linear pipeline** (normalize → ANOVA → PCA → logistic regression): it is the most accurate-per-second method here. With a **real atlas and time to train**, **ProtoCloud** is the most accurate method we benchmarked (0.976 at 49k cells), and the GPU it targets makes its 19× CPU fit cost much less punishing in practice. **scTOP** suits small, low-cardinality problems, where it is by far the cheapest (~1 s) and competitive. **actinn-jax** fits the cases its cost profile is shaped for: when **memory is the binding constraint** (~2× lighter than the linear pipeline at scale, with memory-bounded chunked inference), when a reference is **reused** across many queries (the cached model amortizes the fit), and when the job is the **multi-stage workflow** rather than a single label — broad→refined hand-off, tissue-aware refinement, calibrated abstain, novel-cell-type screening.

For the **broad tier specifically, prefer a purpose-built pan-human model**: **Pan-human Azimuth** (`PAN_HUMAN_AZIMUTH.md`) ships a pretrained hierarchical classifier over a harmonized organism-wide typology (8 levels, 382 leaf types, trained on 9.7M curated cells), with abstention learned rather than thresholded and calibration measured at ECE 0.0044, running at ~1,000 cells/s on a laptop. It is better resourced than our census-built reference and we make no claim to improve on its annotations. What we do with it instead is **use it**: distilling its labels and its hierarchy into actinn-jax produces a tier-1 model of comparable accuracy at roughly nine times its throughput, built without a GPU and without labeled data (§3.5). A curated pan-human model is the right thing to start from; a small fast model is the right thing to iterate with.

The part of any fixed typology that cannot be performed is the **hand-off**: it cannot re-annotate into a user's own focused label set (the HLiCA liver reference, cross-study 0.23/0.58 → 0.72/0.86) or resolve states below a leaf (hepatocyte zonation). ProtoCloud likewise provides uncertainty, gene attribution and a refinement path (fine-tuning a pretrained model). What survives as distinct to actinn-jax is therefore not the breadth of the broad reference but **the chain it enables** — start pan-human, re-annotate into label sets no pretrained model carries, screen what none of them claims — at a cost that keeps every stage on a laptop. The distillation recipe is not specific to this teacher or to humans: it needs a pretrained annotator whose typology can be read off its outputs, so extending the same entry point to other organisms is a matter of finding one (or building the

tier-1 reference from a labeled census slice, as §3.5's census route does).

5 Limitations

- Single hardware family for the in-house panel (Apple Silicon); no discrete-GPU numbers there (deep/foundation tiers would be faster on CUDA — out of scope for the "runs-on-a-laptop" question). The §3.8 controlled run covers the **CPU tier** on one cloud box; the GPU/R methods there are reported from OP's own cloud-CI trace (indicative). A same-hardware GPU-tier run to fold those into a single controlled table is the natural extension.
- **Wall-clock comparisons across methods are only meaningful at matched concurrency, and we learned this the expensive way.** A follow-up run that added our four components to the OP harness (§3.8) used higher task concurrency and produced runtimes that disagree with the controlled run by up to 12× for the same method. Measured `%cpu` across methods spans 49% to 2338% — some are single-threaded, some saturate 23 cores — so any wall-clock ranking silently ranks threading and scheduler contention alongside algorithm. The controlled low-concurrency run is the one we report; the extension establishes only that the components run and what memory they need.
- Subsampled references (to keep the matrix tractable); the scaling section characterizes the size dependence directly.
- Human only; six datasets per benchmark; GPU foundation models beyond scPRINT/UCE (scGPT, Geneformer, popV) not run locally.
- actinn-jax needs more cells/type than linear methods to reach parity (§3.1); on very small references it trails.
- **The distilled tier-1 reference inherits a vocabulary, not a calibration.** Pan-human Azimuth's abstention is trained (and its ECE measured at 0.0044); the distilled student learns hard labels only, so it carries none of that — its confidence barely separates right from wrong (§3.5). Recalibrating it, or distilling from soft targets, is the obvious next step and would need a loss change in the package. Its evaluation is also two withheld *atlases*, not withheld tissue: the census-wide corpus spans 376 tissues, so nothing here measures behaviour on biology the corpus never saw.
- **The gene budget is dataset-dependent, not a free parameter.** More genes help most references but *overfit* a tiny, fine-grained, domain-shifted query (tabula_sapiens −10 pt); our reference-CV + cells-per-class selection rule is validated on 6 datasets with a single clean failure case, so it is directional evidence, not a tuned threshold.
- **Standardization is shipped opt-in**, not default, because it shifts probability calibration that the two-stage abstain thresholds are tuned against; combining the two cleanly would require re-tuning those thresholds.
- **Our classical tier (SVM, kNN, CellTypist) is untuned, and a tuned linear baseline beats actinn-jax.** Souza & Mehta's pipeline (normalize → ANOVA → standardize → PCA(220) → logistic regression) leads the matrix on accuracy (0.839 vs 0.831) and macro-F1 (0.699 vs 0.683) while fitting 4× faster, with its largest margin on the 86-type blood+gut set (+4.2 pt) (`SIMPLE_BASELINES.md`). The §3.1 margins over the classical tier should therefore be read as margins over *untuned* baselines; a like-for-like tuning effort on SVM or CellTypist would likely narrow them further. scTOP is cheaper still on time (~1 s) but competitive only at small, low cardinality — best pbmc macro-F1 (0.837), degrading to 0.649 on liver.
- **actinn-jax's memory advantage is bounded, and the linear pipeline scales further than extrapolation suggests** (`SCALING_MEMORY.md`). Extrapolating the linear pipeline's

small-reference footprint predicts ~32 GB and failure at atlas scale; measured, it uses **13.2 GB** and completes, because `SelectKBest(k=20000)` caps the dense matrix well below the full gene set. The linear/actinn-jax memory ratio is likewise **bounded at ~2–3×** (2.15× at full scale) rather than widening with data, so there is no scaling cliff and no regime tested where the linear pipeline stops being usable. Memory projections in this setting should be measured, not extrapolated.

- **At atlas scale the ordering changes and the deep model leads.** Scaling the lung reference 3k → 49k cells, **ProtoCloud goes from worst (0.722) to best (0.976)**, clear of actinn-jax (0.936) and the linear pipeline (0.939), at 19× the CPU fit cost; scTOP gains nothing from scale (0.819 → 0.846) and loses its memory edge (1.22× actinn-jax at 49k). This replicates on the HLiCA liver atlas (36 types, 525k cells, sweep to 47k ref): same bounded ~2× memory band, same ProtoCloud reversal (0.581 → 0.907), same scTOP stagnation (0.657 → 0.583), with scTOP crossing over to heavier than actinn-jax (1.41×). Conclusions drawn from subsampled references do not transfer to atlas scale, in either direction.
- **Annotation only.** Cross-species transfer and disease-state prediction — the tasks where Souza & Mehta find the largest foundation-model deficits, and where a fast method would be most attractive — are outside this benchmark's scope (human, within/cross-dataset annotation).

Data & code availability

All configs (`configs/paper*.yaml`, including `configs/paper_baselines.yaml` which fills the matrix for the three later-added methods on the identical splits), adapters (`benchmark/adapters/`), result CSVs (`results/paper*/`, `docs/results_*.csv`, unified matrix in `docs/results_paper_matrix_unified.cs`
figure scripts, and the actinn-jax package (github.com/iandriver/actinn-jax) are in these two repositories. Rebuilding the shipped broad reference is a single documented command (`benchmark/explore/update_broad_reference.sh`); the distilled tier-1 reference is built by `distill_dump.py` → `distill_train.py` and ships as `actinn_jax.bundled_reference("panhuman_distill_v1"`

HLiCA data © Edgar et al. 2026 (CC-BY 4.0), [doi:10.64898/2026.06.30.735539]. The distilled reference derives from **Pan-human Azimuth** (Sarkar, Li, Molla, … Satija, bioRxiv 2026, doi:10.64898/2026.07.16.738997); its weights are © the authors under CC BY 4.0, obtained via `panhumanpy` (MIT) and Zenodo. Attribution is a condition of that licence and travels inside the shipped model's `build_info.json`.

References

1. Ma F, Pellegrini M. ACTINN: automated identification of cell types in single cell RNA sequencing. *Bioinformatics* 36(2):533-538 (2020).
2. Abdelaal T, et al. A comparison of automatic cell identification methods for single-cell RNA sequencing data. *Genome Biology* 20:194 (2019).
3. Huang Q, et al. Benchmarking single-cell cell-type annotation methods. *Briefings in Bioinformatics* 25(5):bbae392 (2024).
4. Domínguez Conde C, et al. Cross-tissue immune cell analysis reveals tissue-specific features in humans (CellTypist). *Science* 376:eabl5197 (2022).
5. Aran D, et al. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage (SingleR). *Nature Immunology* 20:163-172 (2019).

6. Kiselev VY, Yiu A, Hemberg M. scmap: projection of single-cell RNA-seq data across data sets. *Nature Methods* 15:359-362 (2018).
7. Xu C, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models (scANVI). *Molecular Systems Biology* 17:e9620 (2021).
8. Lotfollahi M, et al. Mapping single-cell data to reference atlases by transfer learning (scArches). *Nature Biotechnology* 40:121-130 (2022).
9. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. *KDD* 785-794 (2016).
10. Rosen Y, et al. Universal cell embeddings: a foundation model for cell biology (UCE). *Nature* (2026), doi:10.1038/s41586-026-10689-z.
11. Lin Z, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model (ESM-2). *Science* 379:1123-1130 (2023).
12. Open Problems for Single-Cell Analysis Consortium. Open Problems: a living benchmark for single-cell analysis. openproblems.bio (2024).
13. Bradbury J, et al. JAX: composable transformations of Python+NumPy programs (2018). github.com/google/jax.
14. Edgar R, et al. The Human Liver Cell Atlas (HLiCA). doi:10.64898/2026.06.30.735539.
15. Souza H, Mehta P. Parameter-free representations outperform single-cell foundation models on downstream benchmarks. *bioRxiv* (2026), doi:10.64898/2026.02.11.705358.
16. Guo K, Ding J. ProtoCloud: a prototypical self-explaining model for single-cell analysis. *Cell Genomics* 6(6):101217 (2026), doi:10.1016/j.xgen.2026.101217.
17. Yampolskaya M, Souza H, et al. scTOP: cell identity from single-cell data via parameter-free projection. github.com/Emergent-Behaviors-in-Biology/scTOP.
18. Sarkar S, Li Z, Molla G, et al. Organism-scale annotation with Pan-human Azimuth. *bioRxiv* (2026), doi:10.64898/2026.07.16.738997.